# ASSESSOR: a PO-Based WebDriver Test Suites Generator from Selenium IDE Recordings

Maurizio Leotta, Antonio Molinari, Filippo Ricca
Dipartimento di Informatica, Bioingegneria, Robotica e Ingegneria dei Sistemi (DIBRIS), Università di Genova, Italy
maurizio.leotta@unige.it; 4045043@studenti.unige.it; filippo.ricca@unige.it

2022 IEEE Conference on Software Testing, Verification and Validation (ICST) | 978-1-6654-6679-0/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICST53961.2022.00045

***Abstract*—End-to-end automated test scripts are a great way to ensure the quality of web applications, but are often perceived as expensive both during their initial development and subsequent maintenance activities. However, maintenance costs can be reduced when test scripts adopt the Page Object (PO) pattern, a sort of web page facade exposing methods to the test scripts.**

**In this work, we proposed ASSESSOR, a novel tool capable of reducing the effort needed for building PO-based Selenium WebDriver test suites. ASSESSOR allows to simply record the test cases, with only a few additional steps compared to Selenium IDE, and then to automatically generate PO-based WebDriver test suites.**

**The in-depth evaluation performed with four web applications shows that ASSESSOR's adoption allows to reduce the development effort of PO-based web test scripts compared to the classic manual approach: 59% time reduction overall, corresponding to a 2.44 increment in productivity.**

***Index Terms*—End-to-End Testing, Web Testing, Development Effort Reduction, Test Generation, Selenium IDE, Selenium WebDriver.**

## I. INTRODUCTION

It is well known that adopting Test Automation during development is extremely important in real SW projects since it allows to (1) drastically improve the quality of the developed applications and (2) reduce the overall human effort with respect to manual testing [1]. In fact, in the long term, the higher initial development costs required by the development of automated tests are often offset by the (by far) lower re-execution costs: this is particularly true in case the test suites are developed by following appropriate best practices and design patterns [2], [3] that can reduce maintenance costs.

The Page Object (PO) design pattern has emerged in GUI-based test automation as one of the most important patterns for reducing test script maintenance cost and code duplication. Although there is awareness, as well as empirical evidence [4], [5], of the benefits due to the adoption of the PO pattern, the actual burden of the manual implementation of POs and their maintenance is left in the hands of the developers. Existing browser plug-ins used for POs creation, in the context of web apps, are helpful but limited, and frameworks/tools for this purpose, proposed in the literature, are at the prototype stage and difficult to apply in industrial contexts. For example, APOGEN [6], an automated PO tool generator for web apps that we proposed a few years ago, even if almost totally automated, has two main weaknesses: the generation of meaningless names for POs and their methods (a fact that complicates maintenance while the PO-pattern adoption should improve it), and the creation of comprehensive POs, often challenging to manage and use because too complete — in fact, POs generated with APOGEN cover all the functionalities of the entire web app. For these reasons, in this paper, we face the problem of reducing the test suites development effort by employing a different approach which is no longer completely automated (as in the case of APOGEN), but able to support some manual tasks of the Testers in order to minimize the overall costs and, in the end, maximize their productivity.

In recent years, we have started experimenting with the combination of two different approaches (capture-replay and script-based), and related testing frameworks (Selenium IDE [7] and Selenium WebDriver [8]), with the aim of combining their benefits. From the results of our experimentation emerged that a supporting tool was necessary to make the combined approach usable and cost-effective. So, in this paper, we present a tool called ASSESSOR that supports the execution of several steps of our *Hybrid* approach. It exploits the potential of Selenium IDE for recording the tester's actions, and it is capable of automatically generating PO-based Selenium WebDriver test suites starting from the output of Selenium IDE. In the end, high-quality PO-based test suites can be easily generated by ASSESSOR at a cost that is by far lower than manual coding.

To evaluate the increase in productivity of ASSESSOR we have designed and executed a controlled experiment. A professional Tester developed eight PO-based WebDriver test suites for four web applications adopting respectively a classical manual approach to the development and our tool ASSESSOR.

This work extends our preliminary workshop paper [9] in several directions. In our previous paper [9], we sketched the idea of the Hybrid approach without any tool's implementation and performed only a simulation of the potential benefits resulting from its adoption. Instead, here, we have realized the Hybrid approach and described the actual implementation of ASSESSOR. Moreover, we report an empirical evaluation conducted with a professional tester that included the development of 120 test scripts for four web applications to quantify the approach's benefits.

The paper is organized as follows: Section II provides the essential background about capture-replay and script-based testing, in particular introducing the testing frameworks Selenium IDE and Selenium WebDriver and finally sketches the results of the preliminary experiment that led us to conceive

978-1-6654-6679-0/22/$31.00 ©2022 IEEE
DOI 10.1109/ICST53961.2022.00045



Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 11,2022 at 14:41:19 UTC from IEEE Xplore. Restrictions apply.

ASSESSOR. Section III describes all steps of our approach and provides the technical details concerning the implementation of ASSESSOR. Section IV describes the empirical study we carried out for evaluating the benefits from adopting our approach and our tool ASSESSOR, followed by the detailed results reported in Section V. Finally, Section VI reports the related literature Section VII concludes the paper.

## II. BACKGROUND

This section briefly introduces the two main E2E web test scripts development approaches, namely the Capture-Replay testing and Script-based testing. Then, we briefly describe the lessons learnt from an experiment we carried out to evaluate a Hybrid-approach having the goal of combining the two classical approaches to the benefits from the strengths of both.

### A. Capture-replay Web testing

Capture-replay (C&R) web testing tools are based on recording the interaction of the users with the Web application under test and then transform the recordings into test artefacts which can be re-executed several times autonomously. The most advanced C&R testing tools/frameworks, such as for example the Selenium IDE [7] browser's extension, also provide a graphical editor to modify and maintain the recorded test artefacts. Taking Selenium IDE into consideration, we can say that it allows Software testers to record, edit, and debug test artefacts expressed in the Selenese internal language, where a Selenese command is a triple: (command, target, value). Moreover, Selenium IDE is able to automatically generate robust locators [10] for locating the Web elements to interact with during test scripts execution; it can also export Selenese test artefacts and transform them into test scripts written in different languages (e.g., Java) and, finally, it is also able to suggest simple assertions to the tester during the recording.

Creating test artefacts using Selenium IDE is a relatively simple task. Even personnel without coding skills can easily build test suites for complex web applications. However, test artefacts generated using Selenium IDE turn out to be expensive in the long run because they are challenging to maintain [4], [11]. Indeed, registered test artefacts are only valid until there are no changes in the application under test. On the other hand, as soon as the application evolves, the recorded sequences need to be re-recorded, or modified (in this latter case, coding skills could be required), often causing the loss of the advantages gained during the development.

### B. Script-based Web testing

An alternative to C&R testing is the script-based testing. In this case, each test script is manually developed by the tester using an IDE (e.g., Eclipse), a programming language (e.g., Java), and relying on a specific testing framework/library. A notable framework for Web application testing [12] belonging to script-based testing is Selenium WebDriver [8]. Selenium WebDriver extends the chosen programming language with commands to interact with the browser (e.g., click a button and fill a field). Test scripts are then completed with assertions using unit testing frameworks such as JUnit [13]. It is now accepted by practitioners and in the scientific community [4], [11] that test scripts built in this way are also easier to modify and maintain thanks to: (1) the adoption of specific design patterns (e.g., PO design pattern [14]) and, (2) the code modularity. However, the use of Selenium WebDriver also has disadvantages [12]: for example, a significant initial effort necessary to develop a test suite (in particular in case of small test suites [5]) and the need for technical skills proper of a developer (i.e., coding and design pattern knowledge). In script-based testing, the PO design pattern is used for enhancing test maintenance and reducing code duplication [14]. A Page Object is an Object Oriented class that serves as an interface to a Web page. Thus, instead of directly interacting with the Web page, the test scripts call public methods of this PO class whenever needed. The main advantage of this design pattern is that if the Web pages evolve (e.g., with minor changes at the level of layout), the test scripts must not be changed, but only the corresponding POs need to be modified [15].

### C. Combining the benefits of the two approaches. Is it feasible?

Recently, we defined and evaluated a *Hybrid approach* that in principle should be able to combine the benefits of capture-replay (i.e., fast development of test scripts) and script-based (i.e., fast maintenance of test scripts) approaches [4]. Such a Hybrid approach has been realized in a straightforward way that can be easily reproduced with many available (also free) testing tools/frameworks. In brief, testers can use the C&R Selenium IDE testing tool to produce the test artefacts by recording them. Then, they export the artefacts and convert them into well-structured and easier to maintain Selenium WebDriver test scripts, one of the most well-known and used script-based testing framework [16]. However, the results of the experimental study conducted with the help of 20 MSc students [9] were not in favour of that preliminary version of the Hybrid approach (that is fully manual), which proved to be more time-consuming than developing the script-based test suite (adopting the PO-pattern) from scratch. However, the experiment was helpful as it revealed the limitations of the Hybrid approach we implemented and the possibility of improving the results by automating the most expensive phases (measured as the time spent in development). Indeed, an overwhelming effort was required to refactor plain test scripts towards PO-based test scripts in a completely unassisted way: we came to the conclusion that such a step must be automated.

## III. GENERATION OF PO-BASED WEBDRIVER TEST SUITES

Given the encouraging preliminary results obtained with the experiment described in the previous section, we refined the approach and developed a tool called ASSESSOR able to support the approach execution. At the project website (https://sepl.dibris.unige.it/ASSESSOR.php), it is possible to find the link to the source code of ASSESSOR (on GitHub) and a demo video showing its usage.

Indeed, the approach, summarized in Fig. 1, is based on the support given by ASSESSOR to steps 1, 3, and 4. More



Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 11,2022 at 14:41:19 UTC from IEEE Xplore. Restrictions apply.

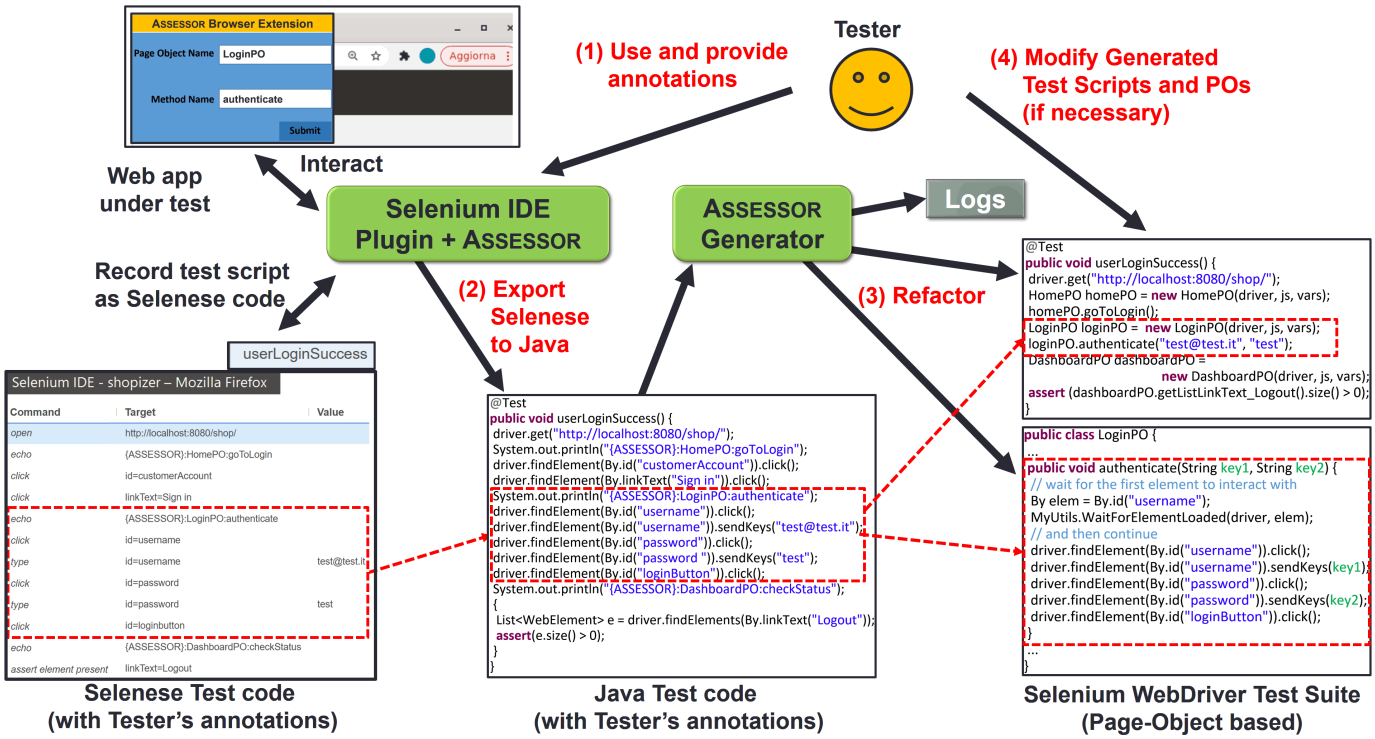



Fig. 1. Approach used for generating PO-based WebDriver test suites with ASSESSOR. During Step (1) ASSESSOR provides a pop-up window used by the Tester to add annotations while recording the test script as Selenese code (saved as echo commands in the Selenese code: e.g., {ASSESSOR}:LoginPO:authenticate). Step (2): the Selenese code is exported by Selenium IDE in Java test code. Step (3): ASSESSOR generates the PO-based test suite using the info contained in the annotations and a log file of the generation (note that in the figure, only one of the generated POs is shown). Step (4): Testers modifies the generated test suite if necessary. The dashed red boxes highlight the code candidate to become a PO method for managing the authenticate operation (from the annotated Selenese to the final PO-based test suite).

in detail, ASSESSOR is capable of (a) helping the Tester to insert the information during test scripts recording (performed with Selenium IDE) helpful for the subsequent Page Objects creation, (b) automating the refactoring phase from plain test code generated with Selenium IDE towards PO-based test scripts, and (c) providing useful info for the final manual refinement (if required). In the continuation of this section, we provide a detailed description of the four steps composing our approach detailing the support given to them by ASSESSOR. Then, we describe the two components of ASSESSOR. They are shown as green boxes in Fig. 1 and consist of a browser extension and a standalone Java program. Additional details on the ASSESSOR implementation and the complete source code of both components can be found on the GitHub repository.

Initially, ***Step (1)***, our approach requires to record the test scripts with Selenium IDE and annotate them. Thus, the Tester has to manually interact with the web application following the test scenarios while Selenium IDE records in Selenese commands all the actions the Tester performs; in practice, this corresponds to the classical usage of Selenium IDE. In the meanwhile, when useful, the Tester can add *annotations*. Annotations are added whenever a new group of logical actions starts so that it is possible to isolate these actions in a PO method once the transformation is done in Selenium WebDriver. For example, a user will have to add an annotation before any authentication on a web application that requires them to fill in the fields of the login form and click on a submit button and then another annotation for the subsequent logical actions group (see {ASSESSOR}:LoginPO:authenticate) in Fig. 1). The ASSESSOR browser extension guides the Tester to insert such annotations in a pop-up window making explicit: (a) to which Page Object each Selenese command that will be subsequently recorded belongs, clearly once transformed in Selenium WebDriver code (e.g., the LoginPO in case of an initial authentication in the login page), and (b) to which PO method each Selenese command that will be recorded belongs (e.g., authenticate(), Fig. 1 show an example of interaction with ASSESSOR for the SHOPIZER web application). Note that in case annotations are misplaced or forgotten, they can always be manually added by editing the Selenese test code.

In ***Step (2)***, the test suite is exported as Java code using the facility included in Selenium IDE. Each test script still has a simple linear structure matching the original Selenese recorded by Selenium IDE: note that no best practices and design patterns are adopted for its generation. Clearly, the Tester's annotations inserted in the previous step (PO and method names) are also exported as Java code (technically, they are inserted in the exported code as print operations, where the string message contains the info about the PO and the method names).

In ***Step (3)***, the annotated Selenium IDE test suite is refactored in a PO-based WebDriver test suite by the Generator module of ASSESSOR. The generator uses the annotations to group the WebDriver Java commands into the expected Page Objects methods and to refactor the test scripts in order to call such methods with the correct parameter values.



Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 11,2022 at 14:41:19 UTC from IEEE Xplore. Restrictions apply.

In the final ***Step (4)***, the Tester analyses the log file produced by ASSESSOR during Step (3) that details all the potential problems encountered during the automated code refactoring. Then, the Tester fixes any issues found by manually modifying the generated code of the PO-based test suite. In rare cases, it may be required to add wait strategies manually (e.g., thread sleeps or explicit waits). This usually happens when a test script attempts to verify the presence of a web element before the browser has completely loaded the page [17]. Note that this limitation derives from the quality of the Java code exported by Selenium IDE, and it is not a limitation of ASSESSOR. The effort required to conduct this manual phase, Step (4), is evaluated with a specific research question in the empirical study evaluating ASSESSOR (see Section IV).

To support the approach above described, ASSESSOR is divided into two components (the green boxes in Fig. 1): the first is a Firefox extension that interacts directly with Selenium IDE, and the other is a Java standalone module implementing the PO-based test suite generator that is able to refactor the plain test code exported using Selenium IDE into test methods and Page Objects without human supervision. Below, more details on the implementation of these components.

**ASSESSOR *browser extension***. To implement the functionality that allows the Tester to insert annotations when recording Selenium IDE test scripts (Step (1)), we had several possible choices. The most straightforward (and probably elegant) solution would have been to modify the Selenium IDE implementation in order to support this functionality. However, in our experience modifying a third-party tool brings several problems such as: (1) Testers should install the modified version of Selenium IDE instead of the original one (and legitimately they cannot trust our implementation); and (2) every time a novel version of Selenium IDE is released, we should re-modify the new version of Selenium IDE by adding our own code. Since we are not a company, our resources are limited, ASSESSOR is still at a prototype level, and we also had a negative experience with a previous implementation of another tool (see the ROBULA+ [18] that became difficult to maintain after just a few releases of the original extended extension called FirePath), we decided to build this module of ASSESSOR as much independent as possible from Selenium IDE. When the ASSESSOR extension is installed in the browser and the Tester brings it in the foreground using the key **F8** (so at request), it shows the pop-up window for inserting the annotation info (see Fig. 1 top-left). This feature is developed in pure JavaScript without any use of libraries or Frameworks like JQuery or Bootstrap. Once the Tester press the submit button in the pop-up, the ASSESSOR extension interacts with Selenium IDE using its standard API and performs the following actions: (1) ask Selenium IDE to insert the annotation in the Selenese code with the details on the PO and method name inserted by the Tester; (2) ask selenium IDE to remove any interaction with the ASSESSOR pop-up windows recorded in the Selenese code (since the main goal of Selenium IDE is to record any action performed, also the actions performed on the ASSESSOR pop-up, that is HTML based, are recorded).

**ASSESSOR *PO-based Test Suite Generator***. ASSESSOR executes step (3) in a totally automated way. It relies on the Java Parser library (https://javaparser.org/) that is used to both read and manipulate each java test file provided in input and to generate the code of the novel PO-based test suite. The PO-based test suite, output of ASSESSOR, is composed of the following elements: a) a single test suite class containing all test scripts (each test script is included in a test method in such class), and b) one or more Page Objects, corresponding to new Java class files. Page Objects are created depending on how many PO-Names are declared in the annotated Selenium IDE Java files composing the input test suite.

For each plain WebDriver test script $t_o$ (*t*est *o*riginal) of the test suite, ASSESSOR proceeds as follows. The initial instructions of $t_o$, such as open the browser and then load the initial URL are copied to the corresponding novel PO-based test script $t_p$ (*t*est *p*age object-based): note that for each $t_o$ a new $t_p$ is generated and saved in the test suite class as a test method. Then, for each PO method declaration found in $t_o$ (i.e., for each annotation inserted by the Tester in $t_o$ during Step (1)), a local variable declaration for the PO $p$ is inserted in $t_p$. It is followed by the call to the method $m$ ($m$ will be inserted in $p$ in the next few sub-steps). For both $p$ and $m$ ASSESSOR uses the names declared by the Tester in the annotation. Then, the PO $p$ is created (if it is not already present) together with the method $m$ (if it is not already present in such PO). Note that details on the possible naming collisions (i.e., when $p$ or $m$ are respectively already present) are given in the following. After that, all the code present in $t_o$ from that point is copied in $m$ until one of the following conditions is true: (1) there is no more instructions in $t_o$; (2) there is a new delimiter for a new PO in $t_o$ (i.e., a new annotation); (3) there is a delimiter that indicates that the instruction in the Page Method is ended (we equipped ASSESSOR of this feature to allow the Tester to leave some instructions in the test, if necessary). At the beginning of the body of $m$ a wait for the first element to interact with is added (useful to reduce flakiness). Then, the code in the body of $m$ is analysed. If such code interacts with web page fields, such as in the case of a form filling, the actual values used are replaced by local variables and the corresponding method parameters are added. In that case, ASSESSOR adds the actual values found in the original method body to the $m$ call in the test $t_p$ (see Fig. 1 for an example with the login credentials). After the method body is generated and all the parameters are set, a final check is performed. ASSESSOR verifies whether there exists another method in the same PO having exactly the same body and parameters. In this case, the new method $m$ method is removed, and the already existing method is used. On the contrary, in case the name of the methods with the same body is different, ASSESSOR logs such method-join operation with a warning message: in fact, in that case, the Tester probably repeated the same list of actions, providing to them different names, while recording other test cases (e.g., the same list of actions is called in one test script recording as "authenticate" and in another "performLogin"). Finally, if two methods have the same name



Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 11,2022 at 14:41:19 UTC from IEEE Xplore. Restrictions apply.

but different bodies, the tool changes the method name by adding a number to the end, and adds the new method to the PO (e.g., authenticate, authenticate_1). This process is repeated for each method declaration found in $t_o$.

ASSESSOR also generates the getter methods for retrieving the values used by the assertions. As in the case of PO method declarations, the Tester has to add an annotation just before the assertion (see the example in Fig. 1). The getter method name used in the assertion is partially derived from the locator used for finding the web element of interest using a heuristic.

If a problem is found at any point in the test suite refactoring process or when some automatic decisions are applied (for example, to resolve method name collisions), a log is produced with a list of warnings and errors, in order to support the Tester during the Step (4). At this point, the Tester can examine the log and have a detailed view of what happened and then proceed, if necessary, with the manual refinements of the created PO-based Selenium WebDriver test suite.

## IV. EMPIRICAL STUDY

This section describes the definition, design, and settings of the experiment we conducted following the template and the guidelines by Wohlin *et al.* [19].

Table I summarizes the main elements of the experiment. The *goal* of the study is to investigate the impact of adopting ASSESSOR on the effort for the development of web test scripts (and thus of web test suites). The *quality focus* regards how the adoption of ASSESSOR affects the time testers employ to develop web test scripts adopting the PO design pattern.

The results of the experiments are interpreted regarding two *perspectives*: (1) researchers interested in empirically evaluating the effects of adopting ASSESSOR and (2) quality managers who want to understand what effort reduction (if any), in terms of time, can be achieved by adopting ASSESSOR.

The *context* of the study is defined as follows: the participant in the experiment is a software professional who is assigned to web test scripts development tasks, while the objects are four open-source web applications.

The research questions of the study are the following:

**RQ$_1$**: *What is the impact of adopting* ASSESSOR *on the development effort of PO-based web test scripts?*

To quantitatively investigate the first research question, we measured the *time* spent by the professional to develop *correct* test cases with and without ASSESSOR for the four assigned web test suites and then compared the difference. We checked the correctness of the implemented test cases by manually inspecting the source code.

TABLE I
OVERVIEW OF THE EXPERIMENT

| | |
|---|---|
| **Goal** | Investigate the impact of adopting ASSESSOR on the development of a PO-based web test suite |
| **Quality Focus** | Effect of the adoption of ASSESSOR on the time testers employ to develop PO-based web test cases. We analyze also the different tasks requiring a different level of skills. |
| **Context** | Objects: four web applications<br>Subject: a professional tester |
| **Null Hypotheses** | No difference in development time |
| **Treatments** | ASSESSOR-Yes, ASSESSOR-No |
| **Dependent Variable** | Time to correctly develop a test script |

**RQ$_2$**: *How much is the fixing time of the test suites created with* ASSESSOR *compared to the manual development time?*

The second research question is relevant since these two activities (i.e., fixing vs. developing from scratch) require a tester to have programming skills. On the other hand, the simple recording can be carried out by testers without programming skills. So this RQ aims at understanding the savings in skilled roles due to the adoption of ASSESSOR. To quantitatively investigate the research question, we compared the *time* spent by the professional to manually develop the test scripts with the one required for fixing the test scripts generated by ASSESSOR.

*Code Quality*: in this study, we do not consider the quality of the generated code. A few years ago, we implemented APOGEN [6], a tool able to generate the Page Objects in a fully automated way. The main weakness of such a tool lies in the quality of the names assigned to the generated Page Objects and methods. With ASSESSOR, we ask the tester to assign such names, so the comprehensibility of the code is very high since they are clearly meaningful. The code inside the PO methods is generally simple to understand since it implements, often with a few commands, the action defined by the method name itself (such as filling a form or clicking a link). Finally, the test scripts are composed of a sequence of calls to PO methods, thus actually very clear to understand.

***Main factor and treatments***. Our experiment has one main factor (ASSESSOR adoption) and two treatments: ASSESSOR-Yes or ASSESSOR-No. In the former case, the professional is asked to implement the PO-based test scripts manually adopting the PO pattern (ASSESSOR-No), while in the latter, the professional is asked to implement the test scripts using ASSESSOR (ASSESSOR-Yes) (i.e., basically recording them in Selenium IDE empowered by ASSESSOR, and then generating the Selenium WebDriver PO-based version with ASSESSOR).

***Objects***. The objects of the study are four web applications belonging to four different domains, namely PET CLINIC[1], SHOPIZER[2], LET'S TRAVEL[3], and RENTAL PROPERTY[4]. In the following, we briefly describe them. Some of these web applications have also been used in other empirical studies (e.g., [6], [20]).

PET CLINIC is a web-based database to record the pet's owner and to keep note of the pet's visit developed with a consolidated framework that is Spring-Boot, as database it uses an H2 memory database. The application used in our experiment is simple to use and straightforward to understand, although non-trivial. PET CLINIC is composed mainly by $1k$ Java LoC and $10k$ XML LoC, excluding libraries (note that

[1]https://github.com/spring-projects/spring-petclinic
[2]https://github.com/shopizer-ecommerce/shopizer
[3]https://github.com/shivaneej/Letstravel
[4]https://github.com/bigprof-software/online-rental-property-manager



Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 11,2022 at 14:41:19 UTC from IEEE Xplore. Restrictions apply.

Spring-Boot offers a variety of built-in annotations and methods allowing to develop web app using just a few lines of code).

SHOPIZER is a web-based e-commerce to handle the buying and selling of objects, developed with Spring-Boot while, as database, it relies on H2. This application, like the previous one, is simple to use and straightforward to understand, although non-trivial. SHOPIZER is composed mainly by $90k$ Java LoC and $5k$ XML LoC, excluding libraries.

RENTAL PROPERTY is a web-based rental management, contact manager, and organizer developed with standard technologies (PHP as server-side language and MySQL as relational DBMS). RENTAL PROPERTY is developed without any server-side framework and for this reason it features $40k$ PHP LoC and $14k$ JavaScript LoC, excluding libraries.

LET'S TRAVEL is a web-based trip planner developed with standard technologies (PHP as server-side language and MySQL as relational DBMS). LET'S TRAVEL is developed without any server-side framework and it features $6.1k$ PHP LoC and 126 JavaScript LoC, excluding libraries.

***Participant***. The experiment was conducted by a professional tester under the supervision of the researchers. He is a full-stack developer and tester having more than ten years of experience in the field. He has good knowledge in developing Selenium IDE and WebDriver test suites. Moreover, before actually performing the experiment, he practiced with ASSESSOR by creating some sample test suites.

***Dependent Variable***. Our experiment has only one dependent variable, which is the time to develop a test case. We used the time to develop a test case as a measure of *productivity* for the test case development task. In order to keep track of the time spent on each test case of the given test suite, we asked the professional to precisely record the time with a granularity at level of seconds (hh:mm:ss). Based on the study definition reported above, we formulated the following null hypothesis to be tested to answer RQ1:

$H_{01}$: *Developing a test case adopting* ASSESSOR *is as efficient as developing a test case using a manual approach.*

Since it is difficult to foresee, a priori, a possible trend, we opted for the most conservative choice and formulated the null hypotheses as a *two-tailed* hypotheses. In fact, on the one hand, developing test cases manually could be faster since there is no overhead associated with the usage of a tool that requires to record first a Selenium IDE version of the test scripts with some additional steps and then it could require to fix the generated Selenium WebDriver PO-based test scripts. On the other hand, using ASSESSOR could make development faster, since the overhead for creating first the Selenium IDE version of the test scripts might be more than balanced by the benefits of generating almost automatically the Selenium WebDriver code (and the fixes could be minimal).

***Analysis Procedure***. After computing descriptive statistics (sufficient to answer RQ2), we used a paired Wilcoxon test [21] to answer RQ1 by comparing the effects of the treatments on the productivity in developing test cases. The objective of a statistical analysis is to reject the null hypotheses above, therefore accepting the corresponding alternative one $H_1$ (stating instead that an effect exists). In all the performed statistical tests, we decided, as it is customary, to accept a probability of 5% of committing Type-I-error ($\alpha$) [19], i.e., rejecting the null hypothesis when it is actually true. While the statistical tests allow checking the presence of significant differences, they do not provide any information about the magnitude of such differences. Therefore, we used the non-parametric Cliff's delta ($|d|$) effect size [22]. The effect size is considered small for $0.148 \leq |d| < 0.33$, medium for $0.33 \leq |d| < 0.474$ and large for $|d| \geq 0.474$.

***Procedure***. We asked the professional tester to implement a set of PO-based test scripts (i.e., a test suite) for the four Web applications described before using: (1) ASSESSOR, i.e., recording Selenium IDE test scripts performing the few additional steps required by our tool, generating the Selenium WebDriver PO-based test suite, and finally fix it when required (ASSESSOR-Yes), and (2) Selenium WebDriver, i.e., adopting a classical manual approach to the development of the PO-based test scripts (ASSESSOR-No).

First, for each web application, we defined a set of 15 test cases covering the main functionalities; they were described with the Gherkin language. Then, we asked the professional to implement them following the two treatments. To balance any learning effect for two applications (namely: LET'S TRAVEL and RENTAL PROPERTY) the professional tester implemented first the test suite using ASSESSOR and then (about 15 days after) using the classic manual approach; on the contrary, in the case of SHOPIZER and PET CLINIC the professional tester adopted a reverse order of implementation (always with a 15 days pause in between). For each test script to implement, the professional tester registered the start and the stop time (format hh:mm:ss): in this way, it was possible to compute precisely the time required to develop it.

## V. RESULTS

In this section, we report the results of the experiment. In particular, for each of the four applications considered, PET CLINIC, SHOPIZER, LET'S TRAVEL, and RENTAL PROPERTY, we report a table (respectively Table II, III, IV, V) containing the following information: the time the professional tester took for developing each test script using or not ASSESSOR and the time required for setting up the test suite and fix general problems affecting multiple test scripts (this happens for the test suites generated by ASSESSOR). To compare the time required to develop each test script adopting a manual approach and ASSESSOR, we proportionally distributed the time spent for these global adjustments over all the test scripts (column overall). Note that since PO methods are often called by several test scripts, this choice is reasonable. Moreover, in the last column of each table, we report the ratio between the time required to develop the test scripts (case ASSESSOR-No) manually and the time needed to develop the test using ASSESSOR (so the case ASSESSOR-Yes): a ratio $> 1$ indicates that developing the test script/suite using ASSESSOR required less time (i.e., the productivity is higher with ASSESSOR) while



Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 11,2022 at 14:41:19 UTC from IEEE Xplore. Restrictions apply.

TABLE II
DETAILED RESULTS PET CLINIC WEB APPLICATION

| # | Description | Manual Development Recorded | Manual Development Overall | Tool Supported Recorded | Tool Supported Overall | Ratio |
|---|---|---|---|---|---|---|
| | Project Creation | 00:01:02 | | 00:01:36 | | |
| | Test Suite Export | | | 00:00:35 | | |
| | Tool Execution | | | 00:00:15 | | |
| | Fix duplicated methods | | | 00:03:19 | | |
| | Fix method addownerFailedTelephoneEmpty | | | 00:00:58 | | |
| | Fix methods variables | | | 00:00:44 | | |
| | Fix methods using the vars construct | | | 00:01:17 | | |
| | | ***00:01:02*** | | ***00:08:44*** | | |
| 1 | 1.1 Add Owner | 00:14:27 | 00:14:38 | 00:01:33 | 00:02:14 | 6.57 |
| 2 | 1.2 Search Owner | 00:03:09 | 00:03:11 | 00:01:06 | 00:01:35 | 2.02 |
| 3 | 1.3 List of Owners | 00:04:17 | 00:04:20 | 00:00:47 | 00:01:08 | 3.85 |
| 4 | 1.4 Edit Owner | 00:11:06 | 00:11:14 | 00:02:07 | 00:03:03 | 3.69 |
| 5 | 1.5 Add Owner failed Address Empty | 00:04:46 | 00:04:49 | 00:01:15 | 00:01:48 | 2.69 |
| 6 | 1.6 Add Pet to owner | 00:13:31 | 00:13:41 | 00:01:33 | 00:02:14 | 6.14 |
| 7 | 1.7 Add a visit to pet | 00:09:09 | 00:09:16 | 00:01:40 | 00:02:24 | 3.87 |
| 8 | 1.8 Add a visit to pet failed | 00:04:05 | 00:04:08 | 00:02:03 | 00:02:57 | 1.40 |
| 9 | 1.9 Search Owner Failed | 00:02:28 | 00:02:30 | 00:00:54 | 00:01:18 | 1.93 |
| 10 | 1.10 Add Owner Failed Telephone Empty | 00:04:56 | 00:05:00 | 00:01:25 | 00:02:02 | 2.45 |
| 11 | 1.11 Add Owner Failed Last Name Empty | 00:02:00 | 00:02:01 | 00:01:18 | 00:01:52 | 1.08 |
| 12 | 1.12 Find All Owner - Owner Exists | 00:02:16 | 00:02:18 | 00:00:49 | 00:01:10 | 1.95 |
| 13 | 1.13 Find All Owner - Owner doesn't exist | 00:02:17 | 00:02:19 | 00:00:48 | 00:01:09 | 2.01 |
| 14 | 1.14 Add Duplicate Pet name | 00:04:31 | 00:04:34 | 00:01:22 | 00:01:58 | 2.33 |
| 15 | 1.15 Add pet to owner failed | 00:01:55 | 00:01:56 | 00:01:19 | 00:01:54 | 1.03 |
| | **Total** | **01:25:55** | **01:25:55** | **00:28:43** | **00:28:43** | 2.99 |

a ratio $< 1$ indicates the contrary (i.e., the productivity is higher when adopting the manual development).

**PET CLINIC**. Concerning the PET CLINIC (see Table II), we can say that the advantages of adopting ASSESSOR are relevant.

Indeed, by looking at the total time required to generate the test suite adopting ASSESSOR, we can see an overall reduction of about three times. Looking at the values of the ratio for each test script, we can see that the results range in the interval 1.03 - 6.57. In several cases, the ratio is higher than 3: this is due to the fact that the test specification requires to interact with several web elements (e.g., filling a form having numerous fields); in these cases, the tester have to spend a lot of time in retrieving the appropriate locator for each field. Then the professional tester has to write the corresponding code interacting with such fields in a PO method, and finally, he can call such a method with the appropriate parameters from the test script. In general, for the majority of the test scripts, the advantage provided by ASSESSOR is in the interval 2-4.

Figure 2 shows the distribution of the times required to develop each test script for the PET CLINIC web application adopting the manual approach (yellow) and ASSESSOR (orange) respectively. By applying a Wilcoxon test (paired analysis), we found that the difference in terms of time is statistically significant, as p-value$<$0.001. The effect size is large given the Cliff's d=0.813.

In the case of SHOPIZER, the time for fixing the generated test scripts is equal to 524s overall. This is by far lower compared to the total development time of 5155s required in the case of the manual approach. Thus, for this web application, ASSESSOR allows reducing the time spent by a tester having programming skills of the 90%.

**SHOPIZER**. Table III summarizes the results for SHOPIZER. In this case, the advantages from adopting ASSESSOR are reduced even if still relevant.

Indeed, by looking at the total time required to generate the test suite suing ASSESSOR we can see a reduction of about 1.7 times. By analyzing in detail the values of the ratio for each test script, we can see that the results range in the interval 0.66 - 5.49. From the table it is evident that for this application, in six cases out of 15 the adoption of ASSESSOR leads to higher test script development times w.r.t. the bare manual development. In the case of SHOPIZER the tester spent

TABLE III
DETAILED RESULTS SHOPIZER WEB APPLICATION

| # | Description | Manual Development Recorded | Manual Development Overall | Tool Supported Recorded | Tool Supported Overall | Ratio |
|---|---|---|---|---|---|---|
| | Project Creation | 00:02:08 | | 00:01:14 | | |
| | Test Suite Export | | | 00:00:26 | | |
| | Tool Execution | | | 00:00:20 | | |
| | Fix duplicated methods | | | 00:03:17 | | |
| | Fix Wait Method | | | 00:22:36 | | |
| | | ***00:02:08*** | | ***00:27:53*** | | |
| 1 | 1.1 - Admin login Successful | 00:08:58 | 00:09:09 | 00:01:15 | 00:02:18 | 3.97 |
| 2 | 1.2 - Admin types a wrong email address | 00:02:29 | 00:02:32 | 00:00:57 | 00:01:45 | 1.45 |
| 3 | 1.3 - Admin types a wrong password | 00:01:05 | 00:01:06 | 00:01:00 | 00:01:51 | 0.60 |
| 4 | 1.4 - Admin logout | 00:02:20 | 00:02:23 | 00:02:07 | 00:03:55 | 0.61 |
| 5 | 2.1 - Admin Create Category | 00:20:07 | 00:20:33 | 00:03:59 | 00:07:21 | 2.79 |
| 6 | 3.1 - Admin Create Manufacturer | 00:10:05 | 00:10:18 | 00:03:25 | 00:06:19 | 1.63 |
| 7 | 4.1 - User Registration Success | 00:08:38 | 00:08:49 | 00:02:16 | 00:04:11 | 2.11 |
| 8 | 4.2 - User Registration Failed | 00:02:48 | 00:02:52 | 00:01:59 | 00:03:40 | 0.78 |
| 9 | 5.1 - User Login Success | 00:14:34 | 00:14:53 | 00:01:28 | 00:02:43 | 5.49 |
| 10 | 5.2 - User Login Failed user | 00:02:23 | 00:02:26 | 00:01:29 | 00:02:44 | 0.89 |
| 11 | 5.3 - User Login Failed pw | 00:02:09 | 00:02:12 | 00:01:25 | 00:02:37 | 0.84 |
| 12 | 5.4 - User Logout Success | 00:07:46 | 00:07:56 | 00:02:32 | 00:04:41 | 1.70 |
| 13 | 6.1 - Change Language | 00:06:08 | 00:06:16 | 00:02:56 | 00:05:25 | 1.16 |
| 14 | 7.1 - Add to cart | 00:04:08 | 00:04:13 | 00:03:28 | 00:06:24 | 0.66 |
| 15 | 7.2 - Remove from cart | 00:06:52 | 00:07:01 | 00:02:40 | 00:04:55 | 1.42 |
| | **Total** | **01:42:38** | **01:42:38** | **01:00:49** | **01:00:49** | 1.69 |

a relevant amount of time in fixing the generated test suite (close to the half time spent using ASSESSOR). This clearly affected all the ratios and led to values under one for several test scripts. In general, for the majority of the test scripts, the advantage provided by ASSESSOR is in the interval 1-2. Figure 2 shows the distribution of the times required to develop each test script for the SHOPIZER web application adopting the manual approach (yellow) and ASSESSOR (orange) respectively. By applying a Wilcoxon test (paired analysis), we found that the difference in terms of time is not statistically significant (even if quite close to significance), as p-value$<$0.064. The effect size is small given the Cliff's d=0.262.

In the case of SHOPIZER, the time for fixing the generated test scripts is equal to 1673s. This is by far lower compared to the total development time of 6158s required in the case of the manual approach. Thus ASSESSOR allows to reduce the time spent by a tester having programming skills of the 73%.

**LET'S TRAVEL**. In the case of LET'S TRAVEL (see Table IV), the results are pretty similar to those of PET CLINIC. Indeed, the advantages of adopting ASSESSOR are, also in this case, relevant. Indeed by looking at the total time required to generate the test suite using ASSESSOR we can see a reduction of about 2.6 times. In this case, by looking at the ratio values for each test script, we can see that the results range in the interval 0.75 - 4.69. In detail, only for one test script, the adoption of ASSESSOR leads to higher time w.r.t. the bare manual development. This can be explained since, by default, ASSESSOR cannot generate a random email address on each session, which leads to pass the test only the first time, and the professional tester had to fix this problem. In general, for the majority of the test scripts, the advantage provided by ASSESSOR is in the interval 1.5-3.5.

Figure 2 shows the distribution of the times required to develop each test script for the LET'S TRAVEL web application adopting the manual approach (yellow) and ASSESSOR (orange) respectively. By applying a Wilcoxon test (paired analysis), we found that the difference in terms of time is statistically significant, as p-value$<$0.001. The effect size is medium given the Cliff's d=0.440.

In the case of LET'S TRAVEL, the time for fixing the generated test scripts is equal to 925s. This is by far lower compared to the total development time of 7388s in the case of the manual approach. Thus ASSESSOR allows to reduce the



Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 11,2022 at 14:41:19 UTC from IEEE Xplore. Restrictions apply.

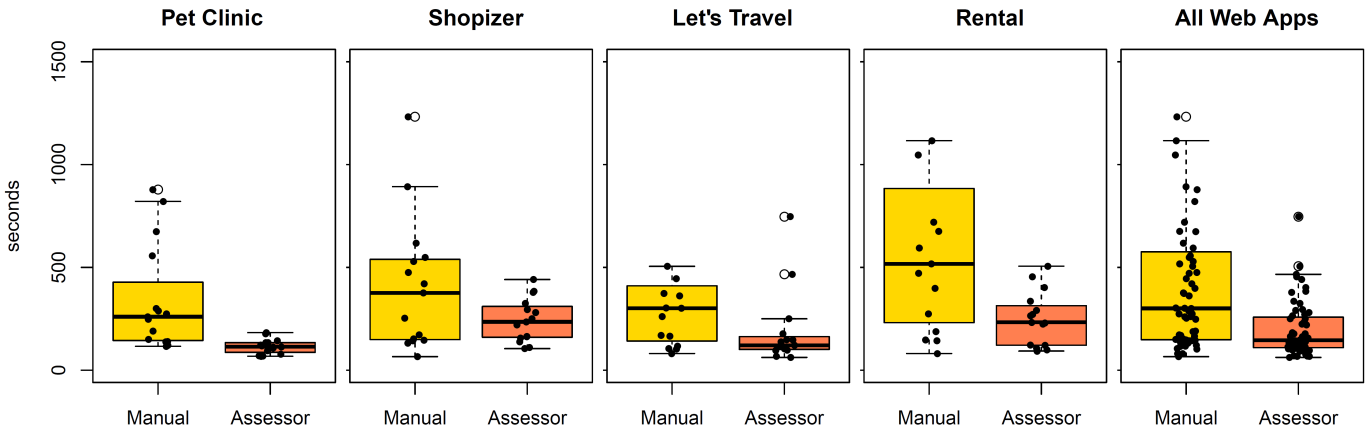



Fig. 2. BoxPlots showing the distribution of the time required (in seconds) to develop the test script adopting a manual approach and ASSESSOR: details for each web application and overall distribution for the four web applications. The few outliers requiring more than 1500s (25min) are not reported.

time spent by a tester having programming skills of the 87% for this web application.

**RENTAL PROPERTY**. Finally, Table V summarizes the results for RENTAL PROPERTY. In this case, the advantages from adopting ASSESSOR reached the best values. Indeed, by looking at the total time required to generate the test suite using ASSESSOR we can see a reduction of about 3.3 times. By analyzing in detail the values of the ratio for each test script, we can see that the results range in the interval 0.79 - 8.74. From the table, it is evident that for this application, only for two cases out of 15, the adoption of ASSESSOR leads to higher test script development time w.r.t. the bare manual development. In general, for the majority of the test scripts the advantage provided by ASSESSOR is in the interval 2-4.

Figure 2 shows the distribution of the times required to develop each test script for the RENTAL PROPERTY web application adopting the manual approach (yellow) and ASSESSOR (orange) respectively. By applying a Wilcoxon test (paired analysis), we found that the difference in terms of time is statistically significant, as p-value=0.001. The effect size is large given the Cliff's d=0.520.

In the case of RENTAL PROPERTY, the time for fixing the generated test scripts is equal to 1203s. This is by far lower compared to the total development time of 12251s in the case of the manual approach. Thus ASSESSOR allows to reduce the time spent by a tester having programming skills of the 90%.

### A. Overall results considering all Web Applications

To conclude, we report in Figure 3 the complete distribution of the ratios associated with each test script developed in the four test suites (corresponding to 60 test scripts in total). From the image, it is evident that a large proportion of the test scripts required less time to be developed using ASSESSOR (see the test scripts having a ratio greater than one, i.e., those at the right of the orange vertical line). More in detail, 9 out of 60 test scripts have a ratio lower than 1 (15% of the test script), while 51 out of 60 have a ratio higher than 1 (i.e., the 85%). Moreover, it is important to note that, for the test scripts having a ratio lower than one the minimum value is 0.6, corresponding to a reduction due to the manual development of the 40% of the development time. On the contrary, a lot of test scripts benefit from much higher values from the adoption of ASSESSOR: in several cases (eight), the time reduction due to the adoption of ASSESSOR is higher than the 75% (i.e., ratio 4).

Figure 2 (last pair of box plots) shows the distribution of the times required to develop each test script aggregating the data of all the web applications considered in this study (also in this case, yellow is for the manual approach and orange for ASSESSOR. By applying a Wilcoxon test (paired analysis), we found that the difference in terms of time is statistically significant, as p-value<0.001. The effect size is medium given the Cliff's d=0.467.

TABLE IV
DETAILED RESULTS LET'S TRAVEL WEB APPLICATION

| # | Description | Manual Recorded | Manual Overall | Development Recorded | Development Overall | Tool Supported Recorded | Tool Supported Overall | Ratio |
|---|---|---|---|---|---|---|---|---|
| | Project Creation | 00:00:50 | | 00:01:25 | | | | |
| | Test Suite Export | | | 00:00:39 | | | | |
| | Tool Execution | | | 00:00:20 | | | | |
| | Fix test adminCreateTrip (upload file) | | | 00:08:52 | | | | |
| | Fix test registerUserSucessfull random email | | | 00:00:37 | | | | |
| | Fix duplicated methods | | | 00:03:32 | | | | |
| | | ***00:00:50*** | | ***00:15:25*** | | | | |
| 1 | 1.1 - Admin login Successful | 00:06:00 | 00:06:02 | 00:01:06 | 00:01:38 | | | 3.71 |
| 2 | 1.2 - Admin types a wrong email address | 00:02:49 | 00:02:50 | 00:00:46 | 00:01:08 | | | 2.50 |
| 3 | 1.3 - Admin types a wrong password | 00:01:20 | 00:01:21 | 00:00:42 | 00:01:02 | | | 1.30 |
| 4 | 1.4 - Admin logout | 00:02:45 | 00:02:46 | 00:01:17 | 00:01:54 | | | 1.46 |
| 5 | 2.1 - Admin Create new Trip | 00:36:14 | 00:36:29 | 00:05:15 | 00:07:46 | | | 4.69 |
| 6 | 2.2 - Admin information trip | 00:05:01 | 00:05:03 | 00:01:10 | 00:01:44 | | | 2.93 |
| 7 | 2.3 - Admin Delete Trip | 00:04:19 | 00:04:21 | 00:01:40 | 00:02:28 | | | 1.76 |
| 8 | 3.1 - User login Successful | 00:06:11 | 00:06:14 | 00:02:00 | 00:02:58 | | | 2.10 |
| 9 | 3.2 - User types a wrong email address | 00:01:42 | 00:01:43 | 00:01:33 | 00:02:18 | | | 0.75 |
| 10 | 3.3 - User types a wrong password | 00:01:56 | 00:01:57 | 00:01:18 | 00:01:55 | | | 1.01 |
| 11 | 3.4 - User logout | 00:04:59 | 00:05:01 | 00:01:22 | 00:02:01 | | | 2.48 |
| 12 | 4.1 - The User change cost for meal pref | 00:31:33 | 00:31:46 | 00:08:25 | 00:12:27 | | | 2.55 |
| 13 | 4.2 - The user is redirect to payment | 00:08:22 | 00:08:25 | 00:02:49 | 00:04:10 | | | 2.02 |
| 14 | 5.1 - Register user successful | 00:07:23 | 00:07:26 | 00:01:39 | 00:02:27 | | | 3.04 |
| 15 | 5.2 - Register user failed duplicate email | 00:01:44 | 00:01:45 | 00:01:05 | 00:01:36 | | | 1.09 |
| | **Total** | **02:03:08** | **02:03:08** | **00:47:32** | **00:47:32** | | | 2.59 |

TABLE V
DETAILED RESULTS RENTAL PROPERTY WEB APPLICATION

| # | Description | Manual Recorded | Manual Overall | Development Recorded | Development Overall | Tool Supported Recorded | Tool Supported Overall | Ratio |
|---|---|---|---|---|---|---|---|---|
| | Project Creation | 00:01:46 | | 00:01:15 | | | | |
| | Test Suite Export | | | 00:00:30 | | | | |
| | Tool Execution | | | 00:00:07 | | | | |
| | Fix duplicated methods | | | 00:01:43 | | | | |
| | Cleanup duplicated Select code | | | 00:01:12 | | | | |
| | Fix Wait Method / Go To | | | 00:15:16 | | | | |
| | | ***00:01:46*** | | ***00:20:03*** | | | | |
| 1 | 1.1 - User Login successfull | 00:07:47 | 00:07:51 | 00:01:03 | 00:01:33 | | | 5.08 |
| 2 | 1.2 - User Login types a wrong username | 00:02:26 | 00:02:27 | 00:01:13 | 00:01:47 | | | 1.37 |
| 3 | 1.3 - User Login types a wrong password | 00:01:20 | 00:01:21 | 00:01:07 | 00:01:39 | | | 0.82 |
| 4 | 1.4 - User Logout | 00:02:23 | 00:02:24 | 00:01:23 | 00:02:02 | | | 1.18 |
| 5 | 2.1 - Create An Application | 00:58:02 | 00:58:32 | 00:04:33 | 00:06:42 | | | 8.74 |
| 6 | 2.2 - Delete an Application | 00:06:36 | 00:06:39 | 00:05:44 | 00:08:26 | | | 0.79 |
| 7 | 3.1 - Show shortcut keyboard | 00:03:06 | 00:03:08 | 00:01:22 | 00:02:01 | | | 1.55 |
| 8 | 4.1 - Create a landlord | 00:39:03 | 00:39:23 | 00:05:09 | 00:07:35 | | | 5.20 |
| 9 | 4.2 - Delete a landlord | 00:11:54 | 00:12:00 | 00:03:00 | 00:04:25 | | | 2.72 |
| 10 | 5.1 - Create a Property | 00:18:26 | 00:18:36 | 00:02:36 | 00:03:50 | | | 4.86 |
| 11 | 5.2 - Delete a Property | 00:09:50 | 00:09:55 | 00:03:48 | 00:05:36 | | | 1.77 |
| 12 | 6.1 - Create unit | 00:17:18 | 00:17:27 | 00:03:18 | 00:04:51 | | | 3.59 |
| 13 | 6.2 - Delete unit | 00:04:32 | 00:04:34 | 00:02:38 | 00:03:53 | | | 1.18 |
| 14 | 7.1 - Create Applicant Tenants | 00:11:09 | 00:11:15 | 00:03:05 | 00:04:32 | | | 2.48 |
| 15 | 7.2 - Delete Applicant tenants | 00:08:33 | 00:08:37 | 00:02:32 | 00:03:44 | | | 2.31 |
| | **Total** | **03:24:11** | **03:24:11** | **01:02:34** | **01:02:34** | | | 3.26 |



Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 11,2022 at 14:41:19 UTC from IEEE Xplore. Restrictions apply.

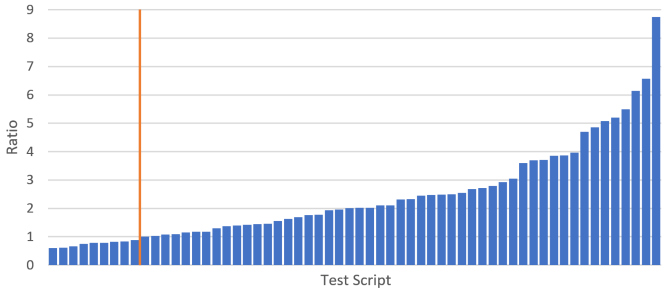


Fig. 3. Distribution of the ratios associated with each test script couple (for the two treatments) of the study (60 in total over four web applications)

To answer **RQ$_1$**: we can state that the positive impact of adopting ASSESSOR on the development effort of PO-based web test scripts is relevant and statistically significant. The overall average value of the ratio considering all the test scripts developed for the four applications is 2.44, corresponding to a reduction in the time required to complete the test suite of the 59%. Only in a few cases, developing the test scripts with ASSESSOR actually increases the developed time.

In conclusion, to answer **RQ$_2$**: we can say that ASSESSOR has a strong positive impact on the time spent by testers having programming skills. Indeed, ASSESSOR allows to drastically reduce the effort required to tester having programming skills (from -73% to -90% in the four analyzed cases). Overall, over the four web applications the time spent in programming tasks is reduced by the 86%. This can lead to relevant savings if testing teams adopting ASSESSOR can employ by far more unskilled testers.

### B. Discussion on the Current Limitations of ASSESSOR

During the recording phase, the professional tester judged the usage of ASSESSOR easy and not invasive since it just requires a few additional steps in addition to a classical C&R tool like Selenium IDE. On the other hand, the generated code has a few problems and fixing them is difficult as developing test scripts manually.

One of the main perceived weaknesses of the current implementation of ASSESSOR includes the lack of an auto-complete feature able to suggest the already defined PO-name and PO-Method (i.e., the ones typed during the recording of previous test scripts belonging to the same test suite). This is a useful feature, since the professional tester had to be very careful to not miss-match or lower/upper-case the name of Page Objects and their methods. These minor errors can be easily fixed after the generation phase but could be avoided if ASSESSOR had already been equipped with such feature. We plan to include this functionality in ASSESSOR by separately storing the sequence of PO and methods defined by the tester during the recording of the entire test suite.

Another limitation concerns the management of the waits required for correctly interacting with the web page elements. Having this feature in ASSESSOR (instead of fixing waits after the generation manually) could help to save a relevant amount of time, especially in dynamic web applications that load data in an asynchronous way (note that in the modern web applications, this is quite common). We plan to include a refined management of the waits in ASSESSOR using the implicit waits mechanism available in the Selenium WebDriver testing framework (as we recently investigated on an industrial case study [23]).

Another limitation concerns the way to interact with the web page elements. Indeed, for example, sometimes clicking more close to an icon rather than clicking more close to the border generated two different identifiers to reach the object and thus two distinct methods in the same PO. We plan to equip ASSESSOR of a post-processing step able to unify different interactions that involve the same conceptual web element.

Finally, a last problem is connected with the interaction with some complex web elements; indeed, the script-based approach could perform direct selections (e.g., a particular instance of a drop-down list), while the recording with a C&R tool requires additional interactions (e.g., click on the drop-down list and then click on the instance of interest). This leads to produce more complicated code, a problem already described in Leotta et al. [24]. Even if this is not an actual limitation, we plan to insert a post processing step able to simply the generated code for such additional commands.

### C. Threats to validity

This section discusses the threats to validity that could affect our results: *internal*, *construct*, *conclusion* and *external* validity threats [19].

*Internal validity* threats concern factors that may affect a dependent variable, in our case, the time required to develop the test scripts. Since the professional tester developed two test suites for each web app, a learning effect might exist. To minimize this problem, we asked the professional to: (1) change the order of development, for two apps first use ASSESSOR while for the other two first use the manual approach; (2) in all cases, waiting at least few days between the development of the two test suites (using different treatments) for the same app. Another possible threat could be related to the textual descriptions of the test cases, which affect the test scripts and the POs to be implemented, and which could give an advantage to a treatment with respect to the other. We mitigated this threat by defining the test cases as usually done in functional testing, i.e., by covering the most relevant functionalities of the web application under test, regardless of the POs that are eventually required for their implementation.

*Construct validity* threats concern the relationship between theory and observation. They are mainly due to how we measured the capability of the professional tester to develop a correct test script and the time needed to do it. Thus, this threat is related to how the correctness of the test script and time were measured. The measurements we conceived to assess the correctness of the test scripts are as objective as possible. Even if we have no guarantee that the implemented test scripts considered in the analysis are correct, executing them and inspecting their implementation gave us a good level of confidence. Moreover, the high skills of the professional tester led to implement test scripts of high quality for both treatments. The time needed to develop the test scripts was



Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 11,2022 at 14:41:19 UTC from IEEE Xplore. Restrictions apply.

recorded with a precision level of seconds (hh:mm:ss), reducing as much as possible inaccuracies.

Threats to *conclusion validity* concern issues that may affect the ability to draw a correct conclusion. They can be due to the number of developed test scripts (120 in total) that may limit the capability of statistical tests to reveal any effect and to the chosen statistical tests.

Threats to *external validity* can be related to: (i) the choice of simple web applications as objects and (ii) the use of only one professional as an experimental participant. The size and complexity of the two web applications are suitable for the time allotted to the experiment but are significantly smaller than that of most industrial web applications. It is possible that some phenomena appear only as the size scales up (both the size of the web application and of the corresponding test suite) and, thus, they are not observable in our experiment. As far as the participant is concerned, we are aware that having more professionals would be preferable. However, finding professionals available to conduct a demanding experiment like the one we designed is not easy.

## VI. Related Work

In the last years, several Chrome and Firefox extensions have been proposed to generate the page objects given their wide usage in practice [25], [26]. For example, among those available for the Chrome browser, we can find: Page Modeller 2.1.0, Page Object Generator by Storm, Selenium Page Object generator 1.3.1, and POM builder. The Page Modeller extension enables testers to scan a web page and generate PO methods for various languages and frameworks (e.g., Selenium WebDriver Java or C# and Robot Framework). Page Object Generator by Storm and Selenium Page Object generator work in a similar way. Given a target web page loaded in the browser, they are able to create a PO for it. The difference between the two extensions is that the latter is more customizable and the generated POs more refined. For example, using Selenium Page Object generator for the Selenium WebDriver framework in Java language, the created POs contain `Findby` annotations and follow the design pattern Page Factory. On the contrary, Page Object Generator by Storm generates a more basic version of POs without employing the Page Factory design pattern. On the other hand, POM builder, even if it has a name that recalls the generation of Page Objects, it only generates robust locators, including XPath, CSS selector for use with Selenium or Protractor. The robustness of locators is another hot topic in Web testing, see, e.g., the following proposals [10], [18], [27]. These browser extensions, although undoubtedly useful for Testers interested in the generation of POs and locators, are, however, very different from Assessor. They just generate the PO of the web page loaded in the browser, while our tool has the goal of generating an entire test suite based on the Page Object pattern and not only POs.

To the best of our knowledge, the first paper addressing the challenging problem of the automated generation of multiple POs for an entire web application is the one by Stocco et al. [6]. In that paper, a tool named APOGEN, based on a crawler able to generate the code for the crawled pages of the target web applications is presented. A component of APOGEN, the clusterer, merges the functionalities exposed by similar pages in a single PO. APOGEN and Assessor have a similar purpose but achieve it differently. APOGEN is based on a crawler and aims to generate the POs for the whole web application, but it does not produce the test scripts, while Assessor is able to generate the POs derived from the registered test scenarios together with the test scripts using such POs. APOGEN has some weaknesses including: the generation of 'meaningless' names for POs and methods (a fact that complicates maintenance), and the creation of comprehensive POs, exposing every functionality of the pages: this can be overwhelming for the Tester since the majority of the code generated maybe not be useful for developing the test scripts of interest (especially when the goal is not to create a test suite having a coverage of the GUI interactions close to the 100%).

Finally, several researchers proposed approaches aimed at generating test scripts automatically, for instance, by applying static and dynamic analysis [20], converting executable specifications [28] and applying model-based testing techniques [29] but none of them generate PO-based test scripts.

## VII. Conclusion and Future Work

In this paper, we described a novel approach supported by the Assessor tool, aimed at reducing the manual development effort of PO-based WebDriver test suites. This tool implements the *Hybrid* approach having the goal of combining the benefits of the two main approaches to end-to-end web test suite development, namely Capture-Replay and Script-based. In this way, it is possible to produce test suites with an excellent code quality (i.e., adopting the PO design pattern and using meaningful names) but developed at a much lower cost compared to that required by classic manual development. As explained in the paper, the tool can manage complex cases and produce advanced parametric Page Object methods to minimize code duplication.

The effectiveness of the approach supported by Assessor has been evaluated with a controlled experiment performed by a professional tester. Results showed that adopting Assessor has a relevant and statistically significant impact on the development effort of PO-based web test scripts (59% time reduction overall, corresponding to a 2.44 increment in productivity w.r.t. manual development). Moreover, most of the manual work required with our approach can be assigned to testers with no programming skills; thus, the reduction in the time spent in tasks involving programming activities, so performed by more skilled testers, reaches the 86%.

In future work, we intend to improve further the implementation of Assessor to reduce the effect of the minor problems encountered during its usage (see Section V-B). Then, we plan to conduct a set of controlled experiments with various students (BSc, MSc, and Ph.D.) to confirm the results obtained with participants having different skills levels.



Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 11,2022 at 14:41:19 UTC from IEEE Xplore. Restrictions apply.

## REFERENCES


[1] M. Fewster and D. Graham, *Software Test Automation: Effective Use of Test Execution Tools*. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 1999.

[2] S. Berner, R. Weber, and R. Keller, "Observations and lessons learned from automated testing," in *Proceedings. 27th International Conference on Software Engineering, 2005. ICSE 2005.*, 2005, pp. 571–579.

[3] V. Garousi and M. Felderer, "Developing, verifying, and maintaining high-quality automated test scripts," *IEEE Softw.*, vol. 33, no. 3, p. 68–75, May 2016. [Online]. Available: https://doi.org/10.1109/MS.2016.30

[4] M. Leotta, D. Clerissi, F. Ricca, and P. Tonella, "Capture-Replay vs. Programmable Web Testing: An Empirical Assessment during Test Case Evolution," in *Proceedings of 20th Working Conference on Reverse Engineering (WCRE 2013)*. IEEE, 2013, pp. 272–281. [Online]. Available: https://doi.org/10.1109/WCRE.2013.6671302

[5] M. Leotta, M. Biagiola, F. Ricca, M. Ceccato, and P. Tonella, "A family of experiments to assess the impact of page object pattern in web test suite development," in *Proceedings of 13th IEEE International Conference on Software Testing, Verification and Validation (ICST 2020)*. IEEE, 2020, pp. 263–273. [Online]. Available: https://doi.org/10.1109/ICST46399.2020.00035

[6] A. Stocco, M. Leotta, F. Ricca, and P. Tonella, "APOGEN: Automatic Page Object Generator for Web Testing," *Software Quality Journal*, vol. 25, no. 3, pp. 1007–1039, Sep. 2017.

[7] Selenium IDE. [Online]. Available: https://www.selenium.dev/selenium-ide/

[8] Selenium WebDriver. [Online]. Available: https://www.selenium.dev/documentation/en/webdriver/

[9] F. Ricca and M. Leotta, "Towards automated generation of PO-based WebDriver test suites from Selenium IDE recordings," in *Proceedings of 12th International Workshop on Automating Test Case Design, Selection, and Evaluation (A-TEST 2021)*. ACM, 2021, pp. 9–16. [Online]. Available: https://doi.org/10.1145/3472672.3473953

[10] M. Leotta, F. Ricca, and P. Tonella, "SIDEREAL: Statistical adaptive generation of robust locators for Web testing," *Journal of Software: Testing, Verification and Reliability (STVR)*, vol. 31, 2021. [Online]. Available: https://doi.org/10.1002/stvr.1767

[11] M. Leotta, D. Clerissi, F. Ricca, and P. Tonella, "Approaches and tools for automated End-to-End web testing," *Advances in Computers*, vol. 101, pp. 193–237, 2016. [Online]. Available: https://doi.org/10.1016/bs.adcom.2015.11.007

[12] B. García, M. Gallego, F. Gortázar, and M. Munoz-Organero, "A survey of the selenium ecosystem," *Electronics*, vol. 9, p. 1067, 06 2020.

[13] jUnit. [Online]. Available: https://junit.org/junit5/

[14] Page Object Models. [Online]. Available: https://www.selenium.dev/documentation/en/guidelines_and_recommendations/page_object_models/

[15] M. Leotta, D. Clerissi, F. Ricca, and C. Spadaro, "Improving test suites maintainability with the page object pattern: an industrial case study," in *Proceedings of the 6th International Conference on Software Testing, Verification and Validation Workshops*, ser. ICSTW 2013. IEEE, 2013, pp. 108–113.

[16] M. Cerioli, M. Leotta, and F. Ricca, "What 5 million job advertisements tell us about testing: a preliminary empirical investigation," in *Proceedings of 35th ACM/SIGAPP Symposium on Applied Computing (SAC 2020)*. ACM, 2020, pp. 1586–1594. [Online]. Available: https://doi.org/10.1145/3341105.3373961

[17] K. Presler-Marshall, E. Horton, S. Heckman, and K. Stolee, "Wait, wait. no, tell me. analyzing selenium configuration effects on test flakiness," in *2019 IEEE/ACM 14th International Workshop on Automation of Software Test (AST)*, 2019, pp. 7–13.

[18] M. Leotta, A. Stocco, F. Ricca, and P. Tonella, "ROBULA+: An algorithm for generating robust XPath locators for web testing," *Journal of Software: Evolution and Process (JSEP)*, vol. 28, no. 3, pp. 177–204, 2016. [Online]. Available: https://doi.org/10.1002/smr.1771

[19] C. Wohlin, P. Runeson, M. Höst, M. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering - An Introduction*. Kluwer Academic Publishers, 2000.

[20] M. Iyama, H. Kirinuki, H. Tanno, and T. Kurabayashi, "Automatically generating test scripts for gui testing," in *2018 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 2018, pp. 146–150.

[21] Wikipedia, "Mann–Whitney U test," last access Sept. 2021. Available at: https://en.wikipedia.org/wiki/Mann-Whitney_U_test.

[22] R. J. Grissom and J. J. Kim, *Effect sizes for research: A broad practical approach*, 2nd ed. Lawrence Earlbaum Associates, 2005.

[23] D. Olianas, M. Leotta, F. Ricca, and L. Villa, "Reducing flakiness in End-to-End test suites: An experience report," in *Proceedings of 14th International Conference on the Quality of Information and Communications Technology (QUATIC 2021)*, ser. CCIS, A. C. R. Paiva, A. R. Cavalli, P. Ventura Martins, and R. Pérez-Castillo, Eds. Springer, 2021, vol. 1439, pp. 3–17. [Online]. Available: https://doi.org/10.1007/978-3-030-85347-1_1

[24] M. Leotta, A. Stocco, F. Ricca, and P. Tonella, "PESTO: Automated migration of DOM-based Web tests towards the visual approach," *Journal of Software: Testing, Verification and Reliability (STVR)*, vol. 28, no. 4, p. e1665, 2018. [Online]. Available: https://doi.org/10.1002/stvr.1665

[25] F. Ricca and A. Stocco, "Web test automation: Insights from the grey literature," in *Proceedings of 47th International Conference on Current Trends in Theory and Practice of Computer Science*, ser. SOFSEM 2021. Springer, 2021.

[26] L. Christophe, R. Stevens, C. De Roover, and W. De Meuter, "Prevalence and maintenance of automated functional tests for web applications," in *2014 IEEE International Conference on Software Maintenance and Evolution*, 2014, pp. 141–150.

[27] V.-L. Nguyen, T. To, and G.-H. Diep, "Generating and selecting resilient and maintainable locators for web automated testing," *Journal of Software: Testing, Verification and Reliability*, vol. 31, no. 3, 2021.

[28] D. Clerissi, M. Leotta, G. Reggio, and F. Ricca, "Towards the generation of end-to-end web test scripts from requirements specifications," in *Proceedings of 25th IEEE International Requirements Engineering Conference Workshops (RE 2017 Workshops)*. IEEE, 2017, pp. 343–350. [Online]. Available: https://doi.org/10.1109/REW.2017.39

[29] H. Tanno and X. Zhang, "Test script generation based on design documents for web application testing," in *2015 IEEE 39th Annual Computer Software and Applications Conference*, vol. 3, 2015, pp. 672–673.




Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 11,2022 at 14:41:19 UTC from IEEE Xplore. Restrictions apply.